Open MCEconomic API

A remote authoritative economy protocol
for Minecraft servers

Protocol version	**1.0**
Status	Specification
Reference client	SUM (Server Utility Mod)
Date	31 July 2026

This document specifies the wire format completely enough that a service can be implemented against it with no access to any client's source. It is deliberately generic: it describes a balance-and-ledger service, not any particular product. Any service satisfying the conformance requirements in Section 12 will interoperate.

Maintained alongside SUM. Markdown source of record: `docs/ECONOMIC_API.md`

Contents

1 Overview **3**
1.1 Design goals . . . 3
1.2 Terminology . . . 3

2 Authority model **3**
2.1 Owned by the service . . . 4
2.2 Owned by the client (the instance) . . . 4
2.3 Consequence . . . 4

3 Deployment topology **4**
3.1 The rule . . . 4
3.2 Why a client must never hold the token . . . 5
3.3 The four scenarios . . . 5
3.4 Single-player and the integrated server . . . 5
3.5 Multiple servers, one economy . . . 6

4 Transport **7**
4.1 Versioning . . . 7
4.2 Standard request headers . . . 7
4.3 Optional integrity headers . . . 8
4.4 Standard response headers . . . 10

5 Authentication and security **10**
5.1 Bearer token (required) . . . 10
5.2 HMAC request signing (optional) . . . 10
5.3 Authorization scope . . . 11
5.4 Transport security . . . 11
5.5 Other requirements . . . 12

6 Common conventions **12**
6.1 Response envelope . . . 12
6.2 Money representation . . . 12
6.3 Identity and accounts . . . 13
6.4 Party objects . . . 14
6.5 Transactions . . . 15
6.6 Idempotency . . . 16
6.7 Errors . . . 17
6.8 Rate limiting . . . 18

7 Endpoints **19**
7.1 `GET /health` [**core**] . . . 19
7.2 `GET /getRequiredHeaders` [**optional**] `headerPolicy` . . . 21
7.3 `POST /resolveAccounts` [**core**] . . . 23
7.4 `GET /getBalance` [**optional**] . . . 23
7.5 `POST /getBalances` [**core**] . . . 23
7.6 `POST /processTransaction` [**core**] . . . 24
7.7 `POST /validateTransaction` [**optional**] `preflight` . . . 26
7.8 `GET /getTransaction` [**core**] . . . 27
7.9 `POST /voidTransaction` [**optional**] `void` . . . 27
7.10 `POST /setBalance` [**optional**] `setBalance` . . . 28

7.11 Holds [optional] `holds` . . . 28
7.12 `GET /getLedger` [optional] `ledger` . . . 29
7.13 `GET /getEvents` [optional] `events` . . . 29
7.14 `GET /getLeaderboard` [optional] `leaderboard` . . . 30

8 Client behaviour **31**
8.1 The game thread must never block . . . 31
8.2 Read path . . . 31
8.3 Write path . . . 31
8.4 Irreversible side effects . . . 32
8.5 Degraded operation . . . 32
8.6 Startup . . . 32

9 Worked flows **32**
9.1 Shop purchase — buyer pays $125.00 . . . 32
9.2 Player transfer with a fee — `/pay Bob 50` at 2% . . . 33
9.3 ATM withdrawal — $100 in bills . . . 33
9.4 Job board — escrow and payout . . . 33
9.5 Administrative adjustment . . . 34

10 Reference implementation notes **34**

11 Client configuration **34**

12 Conformance **35**

13 Change policy **36**

1 Overview

The **Open MCEconomic API** (OMCE) lets a remote HTTP service act as the authoritative owner of a Minecraft player economy. The mod is **always the client**; the remote service is **always the authority**. When the API is enabled, the remote service — not the Minecraft world save — is the source of truth for player balances and the permanent transaction ledger.

This specification is maintained alongside **SUM** (Server Utility Mod), its reference client. SUM is named throughout as the concrete example of what a client does and needs, but the protocol itself is vendor-neutral: nothing on the wire is SUM-specific, and any mod or plugin may implement the client side.

1.1 Design goals

1. **The remote service is authoritative.** The client never invents money. Every balance it shows is a value the service returned, and every change it applies is one the service committed.
2. **Implementable by a modest service.** The required surface is five endpoints. Everything else is an advertised capability the client degrades around.
3. **Safe over an unreliable network.** Minecraft cannot block its tick loop on HTTP. Every mutating call is idempotent and independently recoverable, so a timeout never leaves the question "was the player charged twice?" unanswerable.
4. **Exactly representable amounts.** Money never crosses the wire as a floating-point number.
5. **Forward-compatible.** Clients add economy features regularly. A service written against this document must not need a code change when the client adds a new kind of transaction.

1.2 Terminology

The key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are used as in RFC 2119.

Term	Meaning
Service	The remote HTTP implementation of this API. The authority.
Client	The mod, running on a Minecraft server.
Instance	One Minecraft server talking to the service. Several instances **MAY** share one service.
Account	A settled balance held by the service, identified by an opaque `accountId`.
Party	One side of a transaction. May be an account, or a virtual counterparty.
Settled party	A party whose balance the service actually maintains.
Virtual party	A party the service records for audit but holds no balance for.
Minor unit	The smallest indivisible amount of the currency (e.g. a cent).

2 Authority model

A Minecraft economy is larger than "a number per player". Some of it stays local. This section is normative because it tells a service implementer what they are *not* responsible for.

2.1 Owned by the service

- **Player balances.** The single authoritative number per player.
- **The transaction ledger.** A permanent, ordered, auditable record of every movement.
- **Policy.** Overdraft rules, spending limits, cooldowns, account freezes.
- **Currency definition.** Code, symbol, and scale.

2.2 Owned by the client (the instance)

- **Physical cash.** Bill items are inventory items in the world. The service never sees them; it only sees the balance side of an ATM deposit or withdrawal.
- **Shop escrow.** A shop block accumulates its takings locally until the owner withdraws.
- **Job escrow.** A job listing holds its reward locally between posting and payout.
- **Prices.** Shop prices, plot prices, and job rewards are set in-game and stored in-game.
- **In-world authority.** Block ownership, permissions, and command access.

2.3 Consequence

Only `player` parties need to be settled. A shop, a job listing, a plot, or a pile of cash is a **virtual party**: the service records it as the counterparty on a ledger entry so the books read correctly, but holds no balance for it. A service **MAY** settle other party types (see `settledPartyTypes` in Section 7.1), but **MUST NOT** be required to.

This keeps money conserved end to end. A shop purchase debits the buyer against a virtual `shop` party; the later payout credits the owner *from* that same virtual `shop` party. Net effect across the two transactions: buyer down, owner up, nothing created.

3 Deployment topology

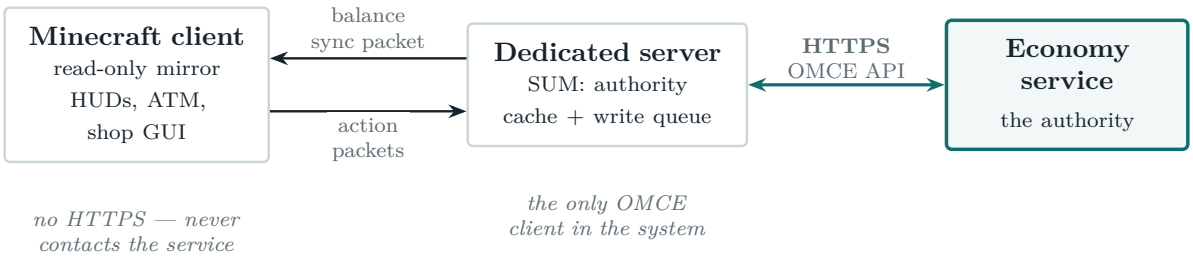
Minecraft has two distinct notions of "client" and "server", and conflating them is the easiest way to build this wrong. This section fixes exactly which process opens a connection to the service.

3.1 The rule

Only the logical server ever contacts the economy service. A Minecraft client never does.

SUM's balances are already server-authoritative. Every mutation happens in a server-side packet handler, command, or tile entity; the resulting balance is pushed to the player's Minecraft client over SUM's own sync packet, and client-side GUIs (wallet HUD, ATM screen, shop GUI, phone app) read that **read-only local mirror**. No GUI has ever read a balance from anywhere but the mirror.

The remote backend slots in underneath that existing arrangement: it replaces where the *server* gets its authoritative number. The client-facing half of the system does not change at all, and gains no network dependency.



Why this matters for the protocol: the service only ever sees connections from a small, known set of Minecraft servers — not from hundreds of player machines. That is what makes a single shared bearer token per instance reasonable, and why rate limits can be modest.

3.2 Why a client must never hold the token

A Minecraft client cannot be trusted with economy credentials:

- Anything shipped to a client is readable by the player who runs it. A token in a modpack's config is a published token.
- A client that could call `/processTransaction` directly could mint itself money, since the service has no way to tell a real client from a modified one.

Operational warning. Modpack configs are distributed to players. Do *not* put `authToken`, `hmacSecret`, or `certificatePassword` in a config that ships with your pack. The client has no use for them — it never opens a connection — so leaving them blank client-side costs nothing. SUM reads all three from `config/sum-secrets.cfg` or the `SUM_ECONOMY_*` environment variables ahead of `sum.cfg`, so a pack can manage `sum.cfg` on the server too while the secrets stay server-only.

3.3 The four scenarios

Scenario	Who runs the authority	Contacts the service?
Dedicated server	The server	**Yes** — the intended deployment
Client connected to a server	Nobody (client mirrors the server)	**No**
Single-player world	The *integrated server* inside the game client	**No**, by default
LAN world ("Open to LAN")	The integrated server of the host	**No**, by default

3.4 Single-player and the integrated server

This is the case that needs an explicit guard, and it is worth stating plainly:

In Minecraft, single-player is not client-only. Opening a single-player world starts a full *integrated server* inside the game client process. All the server-side code — packet handlers, commands, capabilities — runs normally. From SUM's point of view it is a server.

So a player who has a production config on their machine and opens a single-player world would,

without a guard, connect to the shared economy service and spend real balances from a local save. SUM therefore gates the remote backend on the server being **dedicated**:

```
if (server != null && configUsable
        && (server.isDedicatedServer() || allowIntegratedServer)) {
    // construct the remote backend
}
```

`allowIntegratedServer` defaults to **false**. With it off, a single-player or LAN world never contacts the service whatever else is in the config, and falls back to the local economy backends — so the economy still works, as a separate local economy belonging to that save.

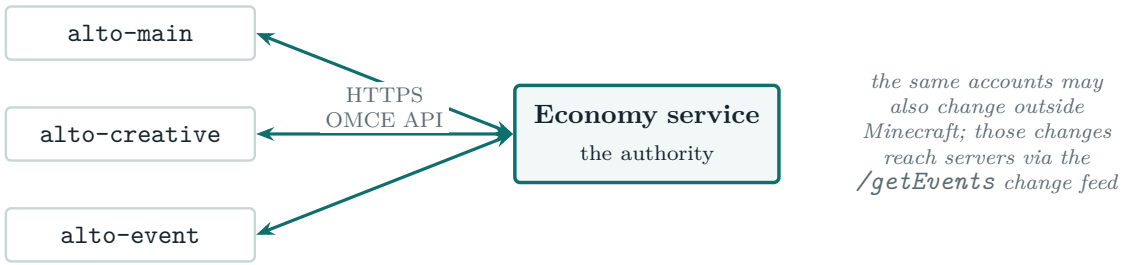


Set `allowIntegratedServer` to `true` only if you deliberately want a single-player world to transact against the live economy — for instance an administrator testing the integration locally. Even then, the request still reports itself as `integrated` via `X-MCE-Environment` (Section 4.2), so the service can apply its own policy.

3.5 Multiple servers, one economy

Several Minecraft servers **MAY** share one service. Each is a distinct **instance** and sends its own `X-MCE-Instance` header, which the service records on every ledger entry so activity can be attributed per server.

Instances do not coordinate with each other. Because the service is the single authority and every balance carries a `version` (Section 7.5), a player who earns money on one server sees it on another as soon as that server's next balance refresh or `/getEvents` poll lands.



Each instance **SHOULD** hold its own bearer token so a single compromised server can be revoked without disturbing the others.

4 Transport

Property	Value
Protocol	HTTP/1.1 or HTTP/2
Transport security	**HTTPS REQUIRED** (Section 5.4)
Base path	`<baseUrl>/api/economic/v1`
Encoding	UTF-8
Request body	`application/json` on every `POST`
Response body	`application/json` on *every* response, including errors

`<baseUrl>` is configured in the client (Section 11) — e.g. `https://economy.example.net`, making the transaction endpoint `https://economy.example.net/api/economic/v1/processTransaction`. The scheme **MUST** be `https` (Section 5.4).

4.1 Versioning

The major version is in the path (`/v1`). Within a major version the service **MUST** remain backward-compatible: it **MAY** add response fields and **MUST** tolerate unknown request fields, but **MUST NOT** remove or repurpose either. A breaking change requires `/v2`.

Both sides also exchange a full version string: the client sends `X-MCE-Protocol-Version: 1.0` on every request, and the service returns `protocolVersion` in every response envelope. If the service cannot serve the requested major version it **MUST** reply `400` with `PROTOCOL_VERSION_UNSUPPORTED`.

4.2 Standard request headers

Header	Required	Description
`Authorization`	Yes	`Bearer <token>`
`Content-Type`	On POST	`application/json; charset=utf-8`
`Accept`	Yes	`application/json`
`X-MCE-Protocol-Version`	Yes	`1.0`
`X-MCE-Instance`	Yes	Instance identifier, e.g. `alto-main`
`X-MCE-Environment`	Yes	`dedicated` or `integrated`; see below
`X-MCE-Request-Id`	Yes	A fresh UUIDv4 per HTTP attempt; log correlation only
`Idempotency-Key`	On mutating calls	See Section 6.6
`X-MCE-Timestamp`	If HMAC enabled	Unix milliseconds
`X-MCE-Signature`	If HMAC enabled	Hex HMAC-SHA256
`User-Agent`	Yes	`<client>/<version> OpenMCEconomicAPI/1.0`

`X-MCE-Request-Id` vs `Idempotency-Key`: the request id changes on every retry; the idempotency key does not. The service keys deduplication off the idempotency key *only*.

X-MCE-Environment

Declares what kind of Minecraft server is calling, **regardless of any client-side setting**:

Value	Meaning
`dedicated`	A standalone Minecraft server process. The normal production case
`integrated`	A server running inside a game client — a single-player or LAN world (Section 3.4)

The client **MUST** send this truthfully on *every* request, deriving it from the actual runtime environment and never from configuration. A client reaching the service from a single-player world (because its operator set `allowIntegratedServer`) still reports `integrated`.

This exists so the service can enforce its own policy rather than trusting the client's. A service **MAY reject** `integrated` requests outright with `403` / `ENVIRONMENT_REJECTED`, **accept but flag** them by recording the environment on the ledger entry, or **ignore** the header entirely — the default for a service that does not care. A service that rejects `integrated` traffic **SHOULD** do so at `/health` as well, so the client discovers it at startup and degrades cleanly rather than failing on a player's first purchase.

4.3 Optional integrity headers

These carry context about the calling Minecraft server that a service can use for anomaly detection and strict policy. SUM sends them by default; they can be disabled with `sendIntegrityHeaders`.

A service **MUST** function correctly when they are absent — **unless it explicitly declares them required**. A service **MAY** require any of them and reject requests that omit them, provided it publishes that policy via `requiredHeaders` in `/health` and `/getRequiredHeaders` (Section 7.2), and rejects with `MISSING_REQUIRED_HEADER`. Requiring a header without publishing it is non-conforming: a client cannot guess.

Header	Example	What it tells the service
`X-MCE-Online-Mode`	`true`	Whether the server authenticates players against Mojang
`X-MCE-World-Id`	`9c1f...`	Stable UUID for the world save, persisted in it
`X-MCE-World-Seq`	`48211`	Monotonic counter in the world save, incremented per transaction
`X-MCE-Session-Id`	`4a2b...`	UUID regenerated on every server boot
`X-MCE-Player-Count`	`14`	Players online at the time of the request
`X-MCE-Initiator-State`	`creative,op`	Risk-relevant state of the initiating player
`X-MCE-Mod-Version`	`2026.07.31`	Version of the mod acting as client

X-MCE-Online-Mode — the one that matters most

A Minecraft server running with `online-mode=false` does **not** authenticate players. Anyone can join using any username and receive that username's UUID. On such a server the Minecraft UUID in a request is an unverified claim, and the identity model of Section 6.3 does not hold.

A service **SHOULD** refuse or heavily restrict traffic from an offline-mode server, because it cannot meaningfully attribute a balance to a person. This is the most valuable of these headers: it is a

genuine statement about whether identity means anything, not a heuristic. Services that accept offline-mode traffic **SHOULD** at minimum record it on every ledger entry.

`X-MCE-World-Id` and `X-MCE-World-Seq` — rollback detection

This pair is the only one with real teeth, because it is a **consistency check against the service's own records** rather than a claim the service must take on faith.

`X-MCE-World-Seq` is a counter stored *in the world save* and incremented on every transaction sent. The service remembers the highest value seen per `X-MCE-World-Id`. Under normal operation it only climbs. If a server restores a world backup, the save reverts — and so does the counter, so the service sees a sequence number at or below one it has already recorded.

> That signature matters because a rolled-back world is the classic Minecraft economy duplication exploit: the world's local state (shop escrow, job listings, items, chest contents) returns to an earlier point while the service's ledger does not. Everything spent in the rolled-back window has been effectively refunded in-world while remaining credited in the ledger.

A service detecting a regression **SHOULD** alert an operator and **MAY** refuse writes from that world until an administrator acknowledges it. It **SHOULD NOT** silently reject, since a legitimate restore after hardware failure produces exactly the same signal and needs a human decision. `X-MCE-Session-Id` distinguishes a restart (new session, counter continues) from a rollback (new session, counter regresses), which is what makes the two separable.

`X-MCE-Initiator-State`

A comma-separated list, empty when nothing applies: `creative`, `spectator`, `op`, `cheats`, `singleplayer`.

`creative` is the notable one: a player in creative mode can spawn items and blocks freely, so selling goods to a shop or an admin-run buy station is effectively minting money. A service **MAY** reject transactions carrying `creative`, or record them for review.

What is deliberately not sent

These headers are intended to be **cheap, non-invasive, and relevant to the transaction at hand**. They describe the *economic trustworthiness of the environment*, not the people in it. The following are excluded from this specification, and a conforming client **MUST NOT** send them:

- Player IP addresses, hostnames, or any network identifiers.
- Hardware, machine, or installation fingerprints.
- The server's own address, MOTD, or port.
- The roster of online players, or any player identity other than the parties to the transaction.
- The installed mod list, or hashes of it.
- The world seed, or world contents.

A service **MUST NOT** require any of these as a condition of service. If a stricter policy is needed than these headers support, the correct control is the token: issue it only to server operators you are willing to trust.

What these headers are not

These are self-reported and therefore not a security boundary. A modified client can send whatever it likes. They exist so an *honest* client can tell the service something useful, and so an *inconsistent* one becomes visible — not to stop a dishonest one.
The real boundary is the bearer token, which never leaves the server operator's hands (Section 3.2). If you do not trust the operator of a Minecraft server, do not issue them a token; no header will compensate.
The exception is the `X-MCE-World-Seq` check, which has value precisely because the service validates it against its own ledger rather than believing it.

4.4 Standard response headers

Header	Required	Description
`Content-Type`	Yes	`application/json; charset=utf-8`
`X-MCE-Protocol-Version`	Yes	The version actually served
`Retry-After`	On `429`/`503`	Seconds, or an HTTP-date
`Deprecation`	No	RFC 8594; set when the endpoint or version is deprecated
`Sunset`	No	RFC 8594 HTTP-date after which it stops working

5 Authentication and security

5.1 Bearer token (required)

Every request carries `Authorization: Bearer <token>`. The token is a shared secret issued by the service operator and configured in the client. Tokens **SHOULD** be per-instance so one compromised Minecraft server can be revoked without disturbing others.

A missing, malformed, or unknown token **MUST** produce `401` with `UNAUTHENTICATED`. A valid token without permission for the requested operation **MUST** produce `403` with `FORBIDDEN`. The token **MUST NOT** appear in service logs or in any response body.

5.2 HMAC request signing (optional)

When the operator enables signing, the client additionally sends `X-MCE-Timestamp` and `X-MCE-Signature`. The signature is computed as:

```
signingString = METHOD + "\n"
              + PATH_WITH_QUERY + "\n"
              + X-MCE-Timestamp + "\n"
              + SHA256_HEX(rawRequestBody)      // SHA256_HEX("") for GET

X-MCE-Signature = HEX(HMAC_SHA256(sharedSecret, signingString))
```

`rawRequestBody` is the exact bytes sent — the service **MUST** verify against the raw body, not a re-serialization of the parsed JSON.

The service **MUST** reject a failing signature (`401` / `SIGNATURE_INVALID`) and **SHOULD** reject a timestamp outside a ±300 second window (`401` / `TIMESTAMP_OUT_OF_RANGE`). The comparison

MUST be constant-time. Signing is an *addition* to the bearer token, never a replacement.

5.3 Authorization scope

Administrative commands call the same endpoints as ordinary gameplay. A service **SHOULD** distinguish them: the client marks such requests with `"initiator": {"role": "admin", ...}`, and the service **MAY** require an elevated token for them, replying `403` / `FORBIDDEN` otherwise.

5.4 Transport security

HTTPS is REQUIRED. Every request carries a bearer token in a header, and every response carries account balances; over plaintext both are readable and forgeable by anything on the network path.

Service requirements

- The service **MUST** serve the API over TLS. TLS 1.2 is the minimum; TLS 1.3 **SHOULD** be preferred.
- The service **SHOULD** redirect or reject plaintext requests rather than serving them. A client **MUST NOT** follow a redirect that downgrades `https` to `http`.

Client requirements

- The client **MUST** reject a `baseUrl` whose scheme is not `https`, refusing to start the economy backend and logging the reason, unless the operator has explicitly set `enableHttp`.
- `enableHttp` defaults to **false** and exists only for local development against a service on `localhost`. When enabled, the client **MUST** log a prominent warning on *every* server startup, not just the first.
- The client **MUST** verify the certificate chain *and* the hostname by default. There is no "disable verification" switch — self-signed and private-CA deployments are supported by supplying the certificate instead.

Self-signed and private-CA certificates

Operators running their own certificate authority, or a self-signed certificate, configure `certificatePath` rather than disabling verification. The value is a path **relative to the client's config file** (absolute paths are also accepted), pointing at the certificate or CA bundle to trust.

Format	Extensions	Notes
PEM	`.pem`, `.crt`, `.cer`	Preferred. One or more `BEGIN CERTIFICATE` blocks.
PKCS#12	`.p12`, `.pfx`	Requires `certificatePassword`.
Java KeyStore	`.jks`	Requires `certificatePassword`.

When `certificatePath` is set, the client builds a trust store containing the platform's default trust anchors **plus** the supplied certificate(s), so a private CA can be added without losing the ability to verify public ones. Hostname verification still applies. An unreadable, malformed, or expired certificate is a fatal configuration error — the client refuses to enable the economy backend rather than silently falling back to an unverified connection.

5.5 Other requirements

- The service **SHOULD** rate limit per token.
- The service **MUST NOT** trust any economic claim in a request beyond the parties, amount, and type — in particular it **MUST** independently validate balance sufficiency. A balance echoed by the client is a cache read, never an assertion.
- The service **MUST** treat player-supplied strings (`playerName`, `reason`, `metadata` values) as untrusted text.

6 Common conventions

6.1 Response envelope

Every response — success or failure, every endpoint — is a JSON object with at least:

```
{
  "ok": true,
  "protocolVersion": "1.0",
  "requestId": "5c9f2a1e-9d1b-4a3f-8a7e-2d5f0b1c3e44",
  "serverTime": "2026-07-31T18:22:04.117Z"
}
```

Field	Type	Required	Description
`ok`	boolean	Yes	`true` on success, `false` on failure
`protocolVersion`	string	Yes	Version served
`requestId`	string	Yes	Echo of `X-MCE-Request-Id`, or service-generated
`serverTime`	string	Yes	RFC 3339 UTC timestamp with milliseconds
`error`	object	When `ok=false`	See Section 6.7

Endpoint-specific fields sit alongside these at the top level. A failure response **MUST NOT** carry endpoint payload fields, with the single documented exception in Section 7.6.

6.2 Money representation

Money MUST NOT cross the wire as a floating-point number. Every amount in this API is a JSON **integer** count of **minor units**.

The service declares its scale once, in `/health`:

```
"currency": {
  "code": "SUM",
  "symbol": "$",
  "minorUnitDigits": 2,
  "displayName": "Dollars"
}
```

`minorUnitDigits` is the power of ten between a minor unit and a whole unit:

minorUnitDigits	1 whole unit =	amount: 12500 means
2	100 minor units	$125.00
0	1 minor unit	$12,500
3	1000 minor units	$12.500

Rules

- `amount` is always a **non-negative** integer. Direction is expressed by `source` and `destination`, never by the sign of `amount`.
- Amounts and balances **MUST** fit in a signed 64-bit integer.
- The service **MUST** reject a non-integer, negative, or out-of-range `amount` with `AMOUNT_INVALID`.
- A `currency` that is not the service's currency code **MUST** be rejected with `CURRENCY_MISMATCH`.

Client-side conversion

SUM stores balances internally as a double-precision dollar value. At the API boundary it converts:

```
amountMinor = round(dollars * 10^minorUnitDigits)     // half-up
dollars     = amountMinor / 10^minorUnitDigits
```

When `minorUnitDigits` is smaller than the client's in-game precision (most importantly `0`, a whole-unit currency), the client **rounds every price up to the next representable unit before charging** and displays the rounded price in-game, so a player is never quoted a price the service cannot settle.

6.3 Identity and accounts

The client knows players by Minecraft UUID and name. The service is not required to use either as its primary key — it may key accounts however it likes. Therefore:

- The client sends **Minecraft identity**; the service resolves it to an account.
- The service returns an **opaque `accountId` string**. The client caches it, echoes it back, and never parses it.
- Player UUIDs are sent canonical, hyphenated, lowercase (`f84c6a79-0a4e-45e0-879b-cd49ebd4c4e2`).
- `playerName` is advisory. Minecraft names change; UUIDs do not. The service **MUST** key off the UUID and **SHOULD** store the most recent name for display.

Unlinked players. If a service requires players to link a Minecraft account before transacting, an unresolved player **MUST** produce `ACCOUNT_NOT_LINKED` — a distinct code from `UNKNOWN_ACCOUNT`, which is reserved for identities the service cannot place at all, such as a malformed `accountId`. A well-formed UUID the service simply has no link for is the former: the player exists and has a route to fixing it.

Such a response **SHOULD** carry a human-readable linking instruction, which the client relays verbatim into chat. It goes in one of two places depending on the response shape, and a service returning unlinked players through both **MUST** populate both:

- **Error envelopes** (`/getBalance`, `/processTransaction`, `/voidTransaction`) — in `error.details.linkInstructi

- **Per-entry lists** (`/resolveAccounts` `accounts[]`, `/getBalances` `unresolved[]`) — as a `linkInstructions` field on the entry itself. These are 200 responses where one bad player must not fail the batch, so there is no error object to hang it from.

Omitting it leaves the player with a bare code and no next step, which is the difference between an ATM that says how to link and one that appears broken.

Auto-provisioning. A service **MAY** create an account on first sight of a UUID, declaring this with the `autoProvisionAccounts` capability.

6.4 Party objects

A **party** is one side of a transaction.

```
{ "type": "player",
  "playerUuid": "f84c6a79-0a4e-45e0-879b-cd49ebd4c4e2",
  "playerName": "SomePlayer",
  "accountId": "acct_10457" }

{ "type": "shop",
  "id": "shop:overworld:120:64:-33",
  "name": "Alex's Emporium",
  "ownerUuid": "f84c6a79-0a4e-45e0-879b-cd49ebd4c4e2" }
```

Field	Type	Required	Description
`type`	string	Yes	Party type; see below
`playerUuid`	string	For `player`	Canonical Minecraft UUID
`playerName`	string	No	Current name, ≤ 32 chars. Advisory
`accountId`	string	No	Previously resolved opaque id
`id`	string	For non-`player`	Stable identifier, ≤ 128 chars
`name`	string	No	Human-readable label, ≤ 64 chars
`ownerUuid`	string	No	Owning player, where one exists. Audit only

Reserved party types

`type`	Settled?	Meaning
`player`	**Always**	A player's account. The only type a service **MUST** settle
`shop`	Virtual	A player-run shop block. `id` encodes world and position
`job`	Virtual	A job-board listing holding escrowed reward
`plot`	Virtual	A land plot being bought or refunded
`cash`	Virtual	Physical bill items in the world; ATM deposits and withdrawals
`system`	Virtual	A money sink or faucet inside the client (fees, rewards, grants)
`external`	Virtual	Something outside the game, for service-originated entries

A service **MUST** accept any of these types and **MUST NOT** reject a transaction merely because a party type is virtual.

Composition rule. Every transaction **MUST** have at least one `player` party. Two `player` parties means a transfer between accounts. One `player` party means money enters or leaves the settled economy. *Zero* `player` parties **MUST** be rejected with `NO_SETTLED_PARTY`, unless the service declares the corresponding type in `settledPartyTypes`.

6.5 Transactions

A transaction is a single atomic movement of `amount` from `source` to `destination`.

Semantics: `source` *pays*; `destination` *receives*. If `source` is a settled account it is debited by `amount`. If `destination` is a settled account it is credited by `amount − fee.amount`.

Transaction types

`type` is a lowercase `snake_case` string. SUM's current vocabulary:

`type`	Source	Destination	Emitted when
`player_transfer`	`player`	`player`	`/pay <player> <amount>`
`atm_withdraw`	`player`	`cash`	Player withdraws bills at an ATM
`atm_deposit`	`cash`	`player`	Player deposits bills at an ATM
`shop_purchase`	`player`	`shop`	Buyer purchases from a shop block
`shop_payout`	`shop`	`player`	Owner withdraws accumulated takings
`job_post_escrow`	`player`	`job`	Poster funds a job listing
`job_payout`	`job`	`player`	Completed job approved
`job_refund`	`job`	`player`	Listing cancelled or expired
`plot_purchase`	`player`	`plot`	Player buys a plot
`plot_refund`	`plot`	`player`	Plot purchase reversed
`loyalty_reward`	`system`	`player`	Playtime or loyalty milestone
`admin_credit`	`system`	`player`	Admin grant
`admin_debit`	`player`	`system`	Admin deduction
`admin_migration`	either	either	Bulk migration between backends

Forward compatibility (important). This vocabulary *will grow*. A service **MUST NOT** reject an unrecognised `type`. It **MUST** record the string verbatim and settle using the `source`/`destination` semantics above, which are complete on their own. A service **MAY** offer a strict mode, but **MUST** default to permissive and **MUST** declare strictness via the `strictTransactionTypes` capability.

Fees

A transaction **MAY** carry a fee, deducted from what the destination receives:

```
"fee": {
  "amount": 250,
  "destination": { "type": "system", "id": "sink.pay_fee" }
}
```

The source is charged the full `amount`; the destination receives `amount − fee.amount`; the fee goes to `fee.destination`, defaulting to `{"type": "system", "id": "sink.fee"}`. `fee.amount` **MUST**

satisfy $0 \leq fee \leq amount$. A service that does not support fees declares `"fees": false`; the client then splits the movement into two transactions.

Initiator

```
"initiator": {
  "playerUuid": "f84c6a79-0a4e-45e0-879b-cd49ebd4c4e2",
  "playerName": "SomePlayer",
  "role": "player"
}
```

`role` is one of `player`, `admin`, `system`. Used for authorization and audit.

Metadata and free text

Field	Limit	Notes
`reason`	$\leq$ 256 chars	Human-readable one-liner, safe for an audit log
`metadata`	$\leq$ 16 keys; keys $\leq$ 64, values $\leq$ 256 chars	Flat string→string map

Both are untrusted, player-influenced text. The service **MUST** store them safely, **MUST NOT** interpret them, and **MUST** reject over-limit values with `MALFORMED_REQUEST` rather than silently truncating.

Transaction status

`status`	Meaning
`committed`	Applied and durable. The only status permitting an irreversible in-game side effect
`pending`	Accepted but not final. The client **MUST** poll `/getTransaction` before acting
`held`	Funds reserved, not yet captured. Only from `/authorizeHold`
`voided`	Reversed by `/voidTransaction`, or a capture that never happened
`rejected`	Not applied. Accompanied by an `error`

6.6 Idempotency

Minecraft servers time out, retry, and restart. Without idempotency, a timed-out purchase would be indistinguishable from a failed one and players would be double-charged.

1. The client **MUST** send an `Idempotency-Key` header on every mutating request. The value is a UUIDv4 generated **once per logical operation** and reused across every retry.
2. `/processTransaction` also carries it in the body as `idempotencyKey`. If both are present and differ, the service **MUST** reject with `MALFORMED_REQUEST`.
3. The service **MUST** persist the key with the full response it produced for **at least 24 hours**.
4. On a repeat with a **matching** body, the service **MUST NOT** apply the operation again. It **MUST** return the **original stored response** with `transaction.replayed` set to `true` and the original HTTP status.

5. On a repeat with a **different** body, the service **MUST** reject with `409` and `IDEMPOTENCY_KEY_REUSED`, applying nothing.
6. If a request with a known key arrives while the first is still in flight, the service **MUST** either block until the first completes and return its response, or reply `409` with `IDEMPOTENCY_IN_PROGRESS` and `retryable: true`.

Body matching **SHOULD** use a canonical hash of the semantically meaningful fields (type, amount, currency, parties, fee), not raw bytes — the client does not guarantee stable JSON key ordering across retries.

Recovery. If the client never receives a response, it retries with the same key. Once retries are exhausted it calls `GET /getTransaction?idempotencyKey=...` to discover the true outcome. A service **MUST** support that lookup — it is how every ambiguous outcome is resolved.

6.7 Errors

```
"error": {
  "code": "INSUFFICIENT_FUNDS",
  "message": "Balance 4200 is below the required 12500.",
  "retryable": false,
  "retryAfterMs": null,
  "details": { "balance": 4200, "required": 12500, "shortfall": 8300 }
}
```

Field	Type	Required	Description
`code`	string	Yes	`SCREAMING_SNAKE_CASE`, from the table below
`message`	string	Yes	English, ≤ 512 chars. Diagnostic, not player-facing
`retryable`	boolean	Yes	Whether retrying the identical request could succeed
`retryAfterMs`	integer	No	Minimum wait before retry
`details`	object	No	Structured, code-specific context

The client maps codes to player-facing chat text itself; `message` goes to the server log. An unknown `code` is treated as `INTERNAL_ERROR` with `retryable` honoured as sent.

Error code table

Code	HTTP	Retryable	Meaning
`UNAUTHENTICATED`	401	No	Missing or bad bearer token
`SIGNATURE_INVALID`	401	No	HMAC verification failed
`TIMESTAMP_OUT_OF_RANGE`	401	No	`X-MCE-Timestamp` outside the window
`FORBIDDEN`	403	No	Token lacks permission
`ENVIRONMENT_REJECTED`	403	No	Refused on environment policy; `details.reason` names the cause
`PROTOCOL_VERSION_UNSUPPORTED`	400	No	Major version not served
`MALFORMED_REQUEST`	400	No	Bad JSON, missing field, or limit exceeded

Code	HTTP	Retryable	Meaning
`MISSING_REQUIRED_HEADER`	400	No	A required header is absent or disallowed; `details.header` names it
`AMOUNT_INVALID`	400	No	Non-integer, negative, or out-of-range amount
`CURRENCY_MISMATCH`	400	No	Not this service's currency
`NO_SETTLED_PARTY`	400	No	Neither party is settleable
`UNSUPPORTED_PARTY_TYPE`	400	No	Party type not recognised (strict services)
`UNSUPPORTED_TRANSACTION_TYPE`	400	No	Type not recognised (strict services)
`UNKNOWN_ACCOUNT`	404	No	No account, and auto-provisioning is off
`ACCOUNT_NOT_LINKED`	409	No	Player must link their account first
`ACCOUNT_FROZEN`	409	No	Account administratively blocked
`INSUFFICIENT_FUNDS`	409	No	Debit would overdraw
`LIMIT_EXCEEDED`	409	Maybe	Policy limit or cooldown; use `retryAfterMs`
`IDEMPOTENCY_KEY_REUSED`	409	No	Key reused with a different body
`IDEMPOTENCY_IN_PROGRESS`	409	**Yes**	First request with this key still running
`TRANSACTION_NOT_FOUND`	404	No	No transaction with that id or key
`NOT_VOIDABLE`	409	No	Too old, or a non-reversible type
`ALREADY_VOIDED`	409	No	Already reversed
`HOLD_EXPIRED`	409	No	Hold lapsed before capture
`HOLD_ALREADY_SETTLED`	409	No	Hold already captured or released
`VERSION_CONFLICT`	409	No	`expectedVersion` did not match
`CURSOR_EXPIRED`	410	No	Event cursor too old to serve
`RATE_LIMITED`	429	**Yes**	Too many requests; honour `Retry-After`
`ECONOMY_READ_ONLY`	503	**Yes**	Reads served, writes suspended
`SERVICE_UNAVAILABLE`	503	**Yes**	Temporarily down
`INTERNAL_ERROR`	500	**Yes**	Unexpected failure

The HTTP status **MUST** agree with the code. The client keys its behaviour off `code` and `retryable`, using the status only for coarse logging.

6.8 Rate limiting

The service **SHOULD** rate limit per token, replying `429` with `RATE_LIMITED` and a `Retry-After` header. A busy 40-player server generates on the order of a few requests per second at peak. The client retries `429` with exponential backoff and full jitter, honouring `Retry-After` as a floor, and **MUST NOT** retry a non-retryable error.

7 Endpoints

Method	Path	Tier
GET	/health	[core]
GET	/getRequiredHeaders	[optional] headerPolicy
POST	/resolveAccounts	[core]
POST	/getBalances	[core]
POST	/processTransaction	[core]
POST	/validateTransaction	[optional] preflight
GET	/getTransaction	[core]
GET	/getBalance	[optional] convenience
POST	/voidTransaction	[optional] void
POST	/setBalance	[optional] setBalance
POST	/authorizeHold	[optional] holds
POST	/captureHold	[optional] holds
POST	/releaseHold	[optional] holds
GET	/getLedger	[optional] ledger
GET	/getEvents	[optional] events
GET	/getLeaderboard	[optional] leaderboard

Why `POST` for some reads? `/getBalances` and `/resolveAccounts` take a list of players. Sixty UUIDs do not belong in a query string, and the client must refresh every online player in one call. These are POST-with-a-body reads: safe, they change nothing, and they carry no `Idempotency-Key`.

7.1 `GET /health` [core]

Liveness and handshake. The client calls this at startup, after any sustained failure, and every `healthPollSeconds`. The response drives every degrade-and-fallback decision.

Request: no body. `Authorization` is **REQUIRED** — the service **MUST NOT** publish capabilities anonymously.

```
{
```

```
  "ok": true, "protocolVersion": "1.0", "requestId": "...",
  "serverTime": "2026-07-31T18:22:04.117Z",
  "status": "ok",
  "implementation": { "name": "example-economy", "version": "3.2.1" },
  "currency": {
    "code": "SUM", "symbol": "$", "minorUnitDigits": 2, "displayName": "Dollars"
  },
  "capabilities": {
    "void": true, "setBalance": true, "holds": false,
    "preflight": true, "headerPolicy": true,
    "ledger": true, "events": true, "leaderboard": true,
    "fees": true, "negativeBalances": false,
    "autoProvisionAccounts": true, "strictTransactionTypes": false,
    "offlinePlayers": true
  },
  "settledPartyTypes": ["player"],
  "requiredHeaders": ["Authorization", "X-MCE-Instance", "X-MCE-Online-Mode"],
  "limits": {
    "maxBatchAccounts": 100,
    "maxTransactionAmount": 100000000,
    "requestsPerMinute": 600,
    "idempotencyRetentionHours": 72
  }
}
```


Field	Type	Required	Description
`status`	string	Yes	`ok`, `degraded`, or `read_only`
`implementation`	object	No	Service name and version, for logging
`currency`	object	Yes	See Section 6.2
`capabilities`	object	Yes	Optional-feature flags; absent means `false`
`settledPartyTypes`	string[]	Yes	Types held balances for; **MUST** include `player`
`requiredHeaders`	string[]	Yes	Headers this service rejects requests without; may be empty
`limits`	object	Yes	See below

`limits` field	Description
`maxBatchAccounts`	Max entries per batch call. **MUST** be ≥ 50
`maxTransactionAmount`	Largest single `amount` accepted, in minor units
`requestsPerMinute`	Advertised rate limit
`idempotencyRetentionHours`	How long keys are remembered. **MUST** be ≥ 24

Capability fallbacks

Flag	If `false`, the client...
`void`	issues a compensating reverse transaction instead
`preflight`	answers affordability from its cached balance only
`headerPolicy`	relies on the `requiredHeaders` summary in `/health` alone
`setBalance`	reads the balance and computes a delta (with a race warning)
`holds`	debits immediately and reverses on failure
`ledger`	omits in-game transaction history UI
`events`	polls `/getBalances` on a timer instead
`leaderboard`	omits any economy leaderboard
`fees`	splits fee-bearing transfers into two transactions
`negativeBalances`	never sends `allowNegative`
`autoProvisionAccounts`	treats `UNKNOWN_ACCOUNT` as permanent per player
`strictTransactionTypes`	(if *true*) warns the operator that new features may break
`offlinePlayers`	avoids transacting for players who are not online

`status: "read_only"` tells the client to serve balances from cache and refuse spending with a clear maintenance message, rather than failing every purchase with a generic error.

7.2 `GET /getRequiredHeaders` [optional] `headerPolicy`

Publishes the service's header policy: which headers it requires, which it merely uses, and which it ignores. A client can call this once at startup and know before its first real request whether it is able to satisfy the service. The same information is summarised in `/health` as `requiredHeaders`, so a minimal client never needs this endpoint; it exists for completeness and for richer, per-endpoint policy.

```
{
  "ok": true, "protocolVersion": "1.0", "requestId": "...", "serverTime": "...",
  "headers": [
    { "name": "Authorization", "requirement": "required", "appliesTo": "*",
      "description": "Bearer token issued by the economy operator." },
    { "name": "Idempotency-Key", "requirement": "required",
      "appliesTo": ["/processTransaction", "/voidTransaction", "/setBalance"],
      "description": "UUIDv4, stable across retries of one logical operation." },
    { "name": "X-MCE-Online-Mode", "requirement": "required", "appliesTo": "*",
      "acceptedValues": ["true"],
      "description": "This economy does not accept offline-mode servers." },
    { "name": "X-MCE-World-Seq", "requirement": "recommended",
      "appliesTo": ["/processTransaction"],
      "description": "Used for rollback detection; requests without it are flagged." },
    { "name": "X-MCE-Player-Count", "requirement": "ignored", "appliesTo": "*" }
  ],
  "policyVersion": "2026-07-31"
}
```

Field	Type	Required	Description
`headers`	array	Yes	One entry per header the service has an opinion about
`headers[].name`	string	Yes	Header name, case-insensitive
`headers[].requirement`	string	Yes	`required`, `recommended`, `optional`, or `ignored`
`headers[].appliesTo`	string | string[]	Yes	`"*"` for all endpoints, or a list of paths
`headers[].acceptedValues`	string[]	No	If present, the value **MUST** be one of these
`headers[].description`	string	No	Human-readable rationale, for diagnostics
`policyVersion`	string	No	Opaque version; changes when the policy does

The list is *not* exhaustive of all headers the service tolerates — anything absent is treated as `optional`. A service **MUST NOT** list a header as `required` that this specification defines as optional *and* that it does not genuinely enforce; the point of the endpoint is that a client can trust it.

Rejecting requests with missing headers

A service **MAY** require any header, including the optional integrity headers of Section 4.3. When a required header is missing or carries a disallowed value, it **MUST** reject with `400` and `MISSING_REQUIRED_HEADER`, naming the header:

```
{
  "ok": false,
  "error": {
    "code": "MISSING_REQUIRED_HEADER",
    "message": "This economy requires X-MCE-Online-Mode: true.",
    "retryable": false,
    "details": { "header": "X-MCE-Online-Mode", "received": "false",
                 "acceptedValues": ["true"] }
  }
}
```

- The service **MUST** apply the same policy to `/health`, so a client that cannot satisfy it discovers this at startup rather than on a player's first purchase.
- The service **MUST NOT** require a header this specification does not define, unless it also publishes it here. A client cannot guess.
- The service **SHOULD** keep the policy stable. Tightening it takes every existing client offline at once.

SUM logs the full header policy at startup when `verboseLogging` is on, and refuses to enable the economy backend with a specific operator-facing error if it cannot satisfy a `required` entry — for example, if the service requires `X-MCE-Online-Mode: true` and the server is in offline mode.

7.3 `POST /resolveAccounts` [core]

Maps Minecraft identities to service accounts. Called on player login and on cache miss. Safe and repeatable; it **MUST NOT** create a transaction.

```
{
  "players": [
    { "playerUuid": "f84c6a79-0a4e-45e0-879b-cd49ebd4c4e2", "playerName": "SomePlayer" },
    { "playerUuid": "0f2b0c33-6f0f-4d1c-9a80-2d3f1b7d9e11", "playerName": "AnotherPlayer" }
  ],
  "createIfMissing": true
}
```

Field	Type	Required	Description
`players`	array	Yes	1 … `maxBatchAccounts` entries
`players[].playerUuid`	string	Yes	Canonical hyphenated UUID
`players[].playerName`	string	No	Current name; the service **SHOULD** store it
`createIfMissing`	boolean	No	Default `false`; honoured only with `autoProvisionAccounts`

```
{
  "ok": true, "protocolVersion": "1.0", "requestId": "...", "serverTime": "...",
  "accounts": [
    { "playerUuid": "f84c6a79-0a4e-45e0-879b-cd49ebd4c4e2",
      "resolved": true, "accountId": "acct_10457",
      "displayName": "SomePlayer", "status": "active", "created": false },
    { "playerUuid": "0f2b0c33-6f0f-4d1c-9a80-2d3f1b7d9e11",
      "resolved": false, "reason": "ACCOUNT_NOT_LINKED",
      "linkInstructions": "Link your Minecraft account before using the economy." }
  ]
}
```

Partial failure is normal here. An unresolvable player is *not* an error response — the call returns `200` with `resolved: false` for that entry. The envelope is `ok: false` only when the whole call failed. `accounts` **MUST** contain exactly one entry per requested player, in request order.

`status` is `active`, `frozen`, or `closed`. The client refuses to spend from a non-`active` account without a round trip.

7.4 `GET /getBalance` [optional]

Single-account convenience read, equivalent to a one-element `/getBalances`.

```
GET /api/economic/v1/getBalance?playerUuid=f84c6a79-0a4e-45e0-879b-cd49ebd4c4e2
```

Exactly one of `playerUuid` or `accountId` is supplied. Returns `404` / `UNKNOWN_ACCOUNT` if there is no such account.

7.5 `POST /getBalances` [core]

Batch balance read — the hot read path. Wallet HUDs, bank HUDs, ATM screens, and shop GUIs all read from the cache this call fills.

```
{
```

```
  "accountIds": ["acct_10457"],
  "playerUuids": ["0f2b0c33-6f0f-4d1c-9a80-2d3f1b7d9e11"]
}
```

At least one of the two **MUST** be non-empty. Combined length **MUST NOT** exceed `maxBatchAccounts`; the service **MUST** reject an oversized batch with `MALFORMED_REQUEST` rather than truncating.

```
{
  "ok": true, "protocolVersion": "1.0", "requestId": "...", "serverTime": "...",
  "balances": [
    { "accountId": "acct_10457",
      "playerUuid": "f84c6a79-0a4e-45e0-879b-cd49ebd4c4e2",
      "balance": 887500, "available": 887500, "held": 0,
      "currency": "SUM", "version": 4271, "status": "active" }
  ],
  "unresolved": [
    { "playerUuid": "0f2b0c33-6f0f-4d1c-9a80-2d3f1b7d9e11",
      "reason": "ACCOUNT_NOT_LINKED",
      "linkInstructions": "Run !mclink 0f2b0c33-... in Discord." }
  ]
}
```

The `version` field (important)

`version` is a **monotonically increasing** integer per account, incremented on every balance change. It is the mechanism that makes an asynchronous client cache correct. The service **MUST** guarantee: if change A commits before change B on the same account, A's `version` is strictly less than B's.

The client applies a balance to its cache *only if* the incoming `version` exceeds the cached one. Without this, a slow `/getBalances` response can arrive after a fast `/processTransaction` response and silently roll the displayed balance backwards.

A service that genuinely cannot maintain a counter **MAY** return the commit timestamp in milliseconds, provided it is strictly increasing per account.

`available` is `balance` − `held`. A service without `holds` returns `available == balance` and `held == 0`.

7.6 `POST /processTransaction` [core]

The heart of the API. Atomically validates and applies one movement of money. **`Idempotency-Key`: REQUIRED.**

```
{
  "idempotencyKey": "9b1f0a0c-1f4e-4d24-8a92-6a70a1c9b7d1",
  "type": "shop_purchase",
  "amount": 12500,
  "currency": "SUM",
  "source": {
    "type": "player",
    "playerUuid": "f84c6a79-0a4e-45e0-879b-cd49ebd4c4e2",
    "playerName": "SomePlayer",
    "accountId": "acct_10457"
  },
  "destination": {
    "type": "shop",
    "id": "shop:overworld:120:64:-33",
    "name": "Alex's Emporium",
    "ownerUuid": "3c2a1b09-77aa-4c11-9d0e-51ab2c9f0011"
```

```
  },
  "fee": null,
  "initiator": {
    "playerUuid": "f84c6a79-0a4e-45e0-879b-cd49ebd4c4e2",
    "playerName": "SomePlayer", "role": "player"
  },
  "reason": "Purchased 16x Diamond",
  "metadata": { "world": "overworld", "item": "minecraft:diamond", "quantity": "16" },
  "occurredAt": "2026-07-31T18:22:03.980Z",
  "instance": "alto-main",
  "allowNegative": false
}
```

Field	Type	Required	Description
`idempotencyKey`	string	Yes	**MUST** equal the `Idempotency-Key` header
`type`	string	Yes	Unknown types **MUST** be accepted
`amount`	integer	Yes	Non-negative minor units
`currency`	string	Yes	**MUST** match the service currency code
`source`	object	Yes	Party that pays
`destination`	object	Yes	Party that receives
`fee`	object	No	Requires the `fees` capability
`initiator`	object	No	Who triggered it
`reason`	string	No	$\leq$ 256 chars
`metadata`	object	No	$\leq$ 16 flat string entries
`occurredAt`	string	No	When the in-game action happened. Advisory
`instance`	string	Yes	**MUST** match `X-MCE-Instance`
`allowNegative`	boolean	No	Honoured only with `negativeBalances` and an admin initiator

Success response

```
{
  "ok": true, "protocolVersion": "1.0", "requestId": "...", "serverTime": "...",
  "transaction": {
    "transactionId": "txn_01J8Z9",
    "status": "committed",
    "type": "shop_purchase",
    "amount": 12500,
    "currency": "SUM",
    "fee": { "amount": 0 },
    "source": { "type": "player", "playerUuid": "f84c...", "accountId": "acct_10457" },
    "destination": { "type": "shop", "id": "shop:overworld:120:64:-33" },
    "postedAt": "2026-07-31T18:22:04.101Z",
    "idempotencyKey": "9b1f0a0c-1f4e-4d24-8a92-6a70a1c9b7d1",
    "replayed": false,
    "instance": "alto-main"
  },
  "balances": [
    { "accountId": "acct_10457",
      "playerUuid": "f84c6a79-0a4e-45e0-879b-cd49ebd4c4e2",
      "balance": 875000, "available": 875000, "held": 0,
      "currency": "SUM", "version": 4272, "status": "active" }
  ]
}
```

`balances` is **REQUIRED** and **MUST** contain the post-transaction balance of every settled party. This is what makes the API pleasant to use: a purchase is one round trip that both moves the money and refreshes the HUD. For a two-player transfer, `balances` has two entries.

Atomicity. The whole transaction **MUST** be all-or-nothing. If the destination cannot be credited, the source **MUST NOT** be debited, and the response **MUST** be a rejection.

Failure response

```
{
  "ok": false, "protocolVersion": "1.0", "requestId": "...", "serverTime": "...",
  "error": {
    "code": "INSUFFICIENT_FUNDS",
    "message": "Balance 4200 is below the required 12500.",
    "retryable": false,
    "details": { "balance": 4200, "required": 12500, "shortfall": 8300 }
  },
  "balances": [
    { "accountId": "acct_10457", "balance": 4200, "available": 4200, "held": 0,
      "currency": "SUM", "version": 4271, "status": "active" }
  ]
}
```

On `INSUFFICIENT_FUNDS` the service **SHOULD** include `balances` even though `ok` is `false` — it lets the client correct a stale cache and tell the player exactly how short they are, in one round trip. This is the one permitted exception to "a failure response carries no payload fields".

7.7 `POST /validateTransaction` [optional] `preflight`

A **dry run** of `/processTransaction`. Takes the identical request body, performs the identical validation, and returns the identical shape of answer — but commits nothing and creates no ledger entry.

This exists so a client can answer "could this succeed?" authoritatively without side effects. SUM uses it to grey out an unaffordable item in a shop GUI, and to check a large plot purchase against service-side spending limits before the player confirms — questions a cached balance cannot answer, because only the service knows its own policy.

`Idempotency-Key`: NOT used. The call is side-effect-free, so replay protection is meaningless; a service **MUST** ignore the header if sent. The request body is identical to `/processTransaction`, and `idempotencyKey` **MAY** be omitted.

```
{
  "ok": true, "protocolVersion": "1.0", "requestId": "...", "serverTime": "...",
  "wouldSucceed": true,
  "balances": [
    { "accountId": "acct_10457", "balance": 887500, "available": 887500,
      "held": 0, "currency": "SUM", "version": 4271, "status": "active" }
  ]
}
```

When the transaction would fail, the call still returns `200` with `ok: true` — the *validation* succeeded, and its answer is "no":

```
{
  "ok": true, "protocolVersion": "1.0", "requestId": "...", "serverTime": "...",
  "wouldSucceed": false,
  "wouldFailWith": {
```

```
    "code": "INSUFFICIENT_FUNDS",
    "message": "Balance 4200 is below the required 12500.",
    "details": { "balance": 4200, "required": 12500, "shortfall": 8300 }
  },
  "balances": [
    { "accountId": "acct_10457", "balance": 4200, "available": 4200,
      "held": 0, "currency": "SUM", "version": 4271, "status": "active" }
  ]
}
```

Field	Type	Required	Description
`wouldSucceed`	boolean	Yes	Whether an identical `/processTransaction` would commit
`wouldFailWith`	object	When `wouldSucceed=false`	The `error` object that call would return
`balances`	array	Yes	Current balances; unchanged by this call

`ok: false` is reserved for the validation itself failing — bad auth, malformed body, service down.

A preflight is advisory, never authorization. The answer can be stale by the time the real call is made: another transaction may land in between, or a limit may trip. The client **MUST** still handle a rejection from `/processTransaction` even after a `wouldSucceed: true`. Preflight improves the interface; it does not replace the authoritative check.

A service **MUST NOT** let this endpoint mutate anything — no balance change, no ledger entry, no idempotency record.

7.8 `GET /getTransaction` [core]

Looks up a transaction by service id or by idempotency key.

```
GET /api/economic/v1/getTransaction?idempotencyKey=9b1f0a0c-1f4e-4d24-8a92-6a70a1c9b7d1
```

This is the recovery path. After a timeout or a server restart mid-purchase, this is how the client learns whether the player was actually charged. A service **MUST** support lookup by `idempotencyKey`.

A `200` returns the same `transaction` object as above, plus `balances` for its settled parties if cheap to compute.

A `404` / `TRANSACTION_NOT_FOUND` is **definitive**: it means the transaction was never applied, and the client is free to retry from scratch. A service **MUST NOT** return `404` for a transaction it has actually committed but merely archived — if lookup is unavailable it **MUST** return `503` / `SERVICE_UNAVAILABLE` instead, so the client keeps waiting rather than double-charging.

7.9 `POST /voidTransaction` [optional] `void`

Reverses a previously committed transaction. Used when an in-game side effect fails after the money moved. `Idempotency-Key`: **REQUIRED.**

```
{
  "idempotencyKey": "3f0e5b21-...",
  "transactionId": "txn_01J8Z9",
  "reason": "Item delivery failed; buyer refunded.",
  "instance": "alto-main"
```

```
}
```

The reversal **MUST** be recorded as its own ledger entry referencing the original via `voidsTransactionId` — the ledger is append-only and the original entry **MUST NOT** be deleted or mutated beyond marking it `voided`.

Errors: `TRANSACTION_NOT_FOUND`, `ALREADY_VOIDED` (which **SHOULD** still return the existing reversal, so a retry converges), `NOT_VOIDABLE`.

When `void` is unavailable, the client instead posts a fresh transaction with `source` and `destination` swapped and `metadata.reverses` set to the original `transactionId`.

7.10 `POST /setBalance` [optional] `setBalance`

Sets an account to an absolute value. Exists because an administrative "set balance to N" is an absolute operation, and expressing it as a delta is racy. `Idempotency-Key:` **REQUIRED.** Requires an admin `initiator`.

```
{
  "idempotencyKey": "b71c...",
  "target": { "type": "player", "playerUuid": "f84c...", "accountId": "acct_10457" },
  "balance": 500000,
  "currency": "SUM",
  "expectedVersion": 4272,
  "initiator": { "playerUuid": "3c2a...", "playerName": "AnAdmin", "role": "admin" },
  "reason": "Manual correction by staff",
  "instance": "alto-main"
}
```

`expectedVersion` is optional optimistic concurrency: if present and it does not match the account's current `version`, the service **MUST** reject with `409` / `VERSION_CONFLICT` and apply nothing. The adjustment **MUST** appear in the ledger as a normal entry with the `admin_set` type, carrying before and after values in `metadata`.

7.11 Holds [optional] `holds`

Two-phase settlement for flows where the client must know funds are good *before* committing an in-world change that is awkward to undo. All three endpoints require an `Idempotency-Key`.

`/authorizeHold` reserves `amount` on the source account: `available` drops, `balance` does not, and `held` rises. `/captureHold` converts a hold into a committed transaction (for `amount` or less). `/releaseHold` cancels it.

```
POST /authorizeHold
{
  "idempotencyKey": "...",
  "type": "plot_purchase",
  "amount": 2500000,
  "currency": "SUM",
  "source": { "type": "player", "playerUuid": "f84c..." },
  "destination": { "type": "plot", "id": "plot:downtown-14" },
  "expiresInSeconds": 120,
  "instance": "alto-main"
}
```

The response carries `hold: {"holdId": "hold_...", "amount": ..., "expiresAt": "...", "status": "held"}` plus `balances`.

`POST /captureHold` takes `{"idempotencyKey", "holdId", "amount"}`, where `amount` is optional, defaults to the full hold, and **MUST NOT** exceed it. `POST /releaseHold` takes `{"idempotencyKey", "holdId", "reason"}`.

Expiry. The service **MUST** release a hold automatically once `expiresAt` passes, and **MUST** reject a late capture with `HOLD_EXPIRED`. The client always sets a short expiry, so a crashed Minecraft server never strands a player's funds.

Where `holds` is unavailable, the client debits immediately and voids on failure — the current behaviour of every SUM economy flow, so holds are a refinement, not a prerequisite.

7.12 `GET /getLedger` [optional] `ledger`

Paged transaction history for one account, backing an in-game statement view.

Parameter	Required	Description
`accountId` *or* `playerUuid`	Yes	Whose history
`limit`	No	1–100, default 25
`cursor`	No	Opaque cursor from a previous `nextCursor`
`since`	No	RFC 3339; entries at or after this time
`until`	No	RFC 3339; entries strictly before this time
`type`	No	Comma-separated transaction types to include

```
{
  "ok": true, "protocolVersion": "1.0", "requestId": "...", "serverTime": "...",
  "entries": [
    { "transactionId": "txn_01J8Z9",
      "type": "shop_purchase",
      "direction": "debit",
      "amount": 12500,
      "currency": "SUM",
      "balanceAfter": 875000,
      "counterparty": { "type": "shop", "id": "shop:overworld:120:64:-33",
                        "name": "Alex's Emporium" },
      "reason": "Purchased 16x Diamond",
      "postedAt": "2026-07-31T18:22:04.101Z",
      "status": "committed" }
  ],
  "nextCursor": "eyJvIjoxMjM0fQ",
  "hasMore": true
}
```

`direction` is `debit` or `credit` *from the perspective of the queried account*, so the client need not work out which side it was on. Entries **MUST** be newest-first. `nextCursor` is `null` when `hasMore` is `false`.

7.13 `GET /getEvents` [optional] `events`

An ordered change feed. This is how balance changes that happen **outside Minecraft** reach the game — a payout, an admin correction, interest, anything the service does on its own.

A change feed is used rather than webhooks deliberately: a Minecraft server is usually behind NAT with no inbound reachability, so it must pull.

Parameter	Required	Description
`cursor`	No	Resume point. Omit on first call to start from *now*
`limit`	No	1–500, default 100
`playerUuids`	No	Comma-separated filter. Omit for all accounts


```
{
  "ok": true, "protocolVersion": "1.0", "requestId": "...", "serverTime": "...",
  "events": [
    { "cursor": "evt_00000000000129",
      "kind": "balance_changed",
      "accountId": "acct_10457",
      "playerUuid": "f84c6a79-0a4e-45e0-879b-cd49ebd4c4e2",
      "balance": 900000, "available": 900000, "held": 0,
      "version": 4274,
      "occurredAt": "2026-07-31T18:31:12.004Z",
      "transaction": { "transactionId": "txn_01J8ZK", "type": "external_credit",
                       "amount": 25000, "reason": "Weekly payout" } }
  ],
  "nextCursor": "evt_00000000000129",
  "hasMore": false
}
```


`kind`	Meaning
`balance_changed`	An account's balance moved; carries the new authoritative balance
`account_status_changed`	An account was frozen, unfrozen, or closed
`account_linked`	A previously unlinked player now has an account

Requirements

- Cursors **MUST** be opaque, totally ordered, and stable. A client that stores `nextCursor` and presents it later **MUST** receive every event after it, exactly once, in order.
- The service **MUST** retain events for at least **24 hours** so a Minecraft server can restart without losing changes.
- If a cursor is too old to serve, the service **MUST** reply `410 Gone` with `CURSOR_EXPIRED`. The client then discards its cache and re-reads every online player.
- An empty feed returns `events: []` with the cursor unchanged. The service **MAY** long-poll but **MUST** return within 30 seconds regardless.

Because every event carries `version`, the client can merge the feed with `/processTransaction` responses safely — whichever arrives second, only the higher `version` wins.

7.14 `GET /getLeaderboard` [optional] `leaderboard`

Richest accounts, for an in-game "baltop". Parameters: `limit` (1–100, default 10), `offset` (default 0), `minecraftOnly` (restrict to accounts with a linked UUID).


```
{
  "ok": true, "protocolVersion": "1.0", "requestId": "...", "serverTime": "...",
  "entries": [
    { "rank": 1, "accountId": "acct_10457",
```

```
      "playerUuid": "f84c6a79-0a4e-45e0-879b-cd49ebd4c4e2",
      "displayName": "SomePlayer", "balance": 887500 }
  ],
  "totalAccounts": 214
}
```

A service **MAY** omit `playerUuid` for accounts with no linked Minecraft identity, and **MAY** exclude accounts that have opted out of public ranking.

8 Client behaviour

This section is normative for the *client*. A service implementer can skip it, but reading it explains why the API is shaped the way it is.

8.1 The game thread must never block

Minecraft runs its world on a single thread at 20 ticks per second. A 200 ms HTTP call on that thread is a four-tick freeze for every player.

- *All* HTTP occurs on a dedicated background executor.
- All reads on the game thread are served from an in-memory cache.
- All writes are enqueued, applied optimistically, and reconciled when the response lands.

8.2 Read path

1. On player login, call `/resolveAccounts` then `/getBalances` for that player.
2. Populate a cache keyed by player UUID holding `accountId`, `balance`, `version`, and `status`.
3. Every in-game balance read hits the cache. It never touches the network.
4. Refresh the cache by, in order of preference: `/processTransaction` responses, the `/getEvents` feed, and a periodic `/getBalances` sweep of online players.
5. **Apply a balance only if its `version` exceeds the cached `version`.** Out-of-order responses are discarded, not applied.

8.3 Write path

1. Generate one `Idempotency-Key` (UUIDv4) for the logical operation.
2. Check the cached balance for an obvious rejection and fail fast in-game. *This is a courtesy check, not authorization* — the service revalidates and its answer wins.
3. Apply an optimistic delta to the cache so the HUD responds instantly.
4. Enqueue `POST /processTransaction`.
5. On `committed`: replace the cached balance with the authoritative one.
6. On rejection: roll back the optimistic delta, restore the authoritative balance if the response carried one, and show a message derived from `error.code`.

7. On timeout or transport failure: retry with the *same* key, up to `maxRetries`, with exponential backoff and jitter. Then call `/getTransaction?idempotencyKey=...`. Only a definitive `404` / `TRANSACTION_NOT_FOUND` permits treating the operation as not applied.

8.4 Irreversible side effects

Any in-world effect that cannot be cleanly undone **MUST** wait for `status: "committed"`: handing a player physical bills, transferring purchased items, assigning plot ownership. The client shows a brief "processing…" state rather than acting optimistically. Effects that *are* cheaply reversible (a HUD number, a chat line) may be applied optimistically.

Where the service supports `holds`, the client prefers authorize → perform effect → capture, removing the failure window entirely.

8.5 Degraded operation

When `/health` fails, or write calls fail persistently, the client enters degraded mode and follows the configured `unavailablePolicy`:

Policy	Behaviour
`deny` *(default)*	Balances shown from cache, marked stale. All spending refused with a clear message. **No money moves while the authority is unreachable.**
`cached`	Reads from cache; writes queued in memory and flushed on recovery, up to a bounded queue. Beyond the bound, behaves as `deny`.
`local`	Falls back to the client's built-in local balances. **Balances will diverge** and require manual reconciliation. Single-player and offline testing only.

8.6 Startup

1. Read config. If the feature is disabled, the remote backend is never constructed.
2. Validate the transport: reject a non-`https` `baseUrl` unless `enableHttp` is set, and load `certificatePath` if configured. A transport that cannot be established securely is a fatal configuration error, not a fallback to an unverified connection.
3. Call `/health`. Record currency, capabilities, and limits.
4. Log one line summarising the negotiated setup — currency scale, available optional features, and any capability that will force a fallback.
5. If `/health` fails, apply `unavailablePolicy` and retry with backoff.

In SUM the remote backend, when enabled and healthy, takes priority over the existing `EconomyBridge` backends. The precedence becomes: **remote API → EconomyInc → SUM local capability**.

9 Worked flows

Each corresponds to an existing SUM feature. Amounts assume `minorUnitDigits: 2`.

9.1 Shop purchase — buyer pays $125.00

One transaction. The shop is virtual; the client keeps the takings in the shop block until the owner withdraws.

```
POST /processTransaction
{ "type": "shop_purchase", "amount": 12500,
  "source":      { "type": "player", "playerUuid": "buyer-uuid" },
  "destination": { "type": "shop", "id": "shop:overworld:120:64:-33",
                   "ownerUuid": "owner-uuid" } }
  -> committed, balances[buyer].balance = 875000
```

The client then transfers the item. If that fails it voids (`POST /voidTransaction`). Later, the owner empties the till — a second transaction, in the opposite direction:

```
POST /processTransaction
{ "type": "shop_payout", "amount": 12500,
  "source":      { "type": "shop", "id": "shop:overworld:120:64:-33" },
  "destination": { "type": "player", "playerUuid": "owner-uuid" } }
```

Net across both: buyer −12500, owner +12500. Money conserved.

9.2 Player transfer with a fee — `/pay Bob 50` at 2%

SUM's `/pay` charges the sender the full amount and credits the recipient the amount minus the configured fee, which vanishes as a money sink. With the `fees` capability that is one call:

```
POST /processTransaction
{ "type": "player_transfer", "amount": 5000,
  "source":      { "type": "player", "playerUuid": "alice-uuid" },
  "destination": { "type": "player", "playerUuid": "bob-uuid" },
  "fee": { "amount": 100,
           "destination": { "type": "system", "id": "sink.pay_fee" } } }
  -> committed; alice -5000, bob +4900
```

Without `fees`, the client sends `player_transfer` for 4900 (alice → bob) followed by `player_transfer` for 100 (alice → `system:sink.pay_fee`), voiding the first if the second fails.

9.3 ATM withdrawal — $100 in bills

Money leaves the settled economy and becomes an item. **The bill is issued only after `committed`.**

```
POST /processTransaction
{ "type": "atm_withdraw", "amount": 10000,
  "source":      { "type": "player", "playerUuid": "player-uuid" },
  "destination": { "type": "cash", "id": "atm:overworld:88:70:12" } }
  -> committed  ==> client gives the player a $100 bill item
  -> rejected   ==> client gives nothing and shows the error
```

The deposit direction is the mirror image: the client consumes the bill items first, then posts `atm_deposit` from `cash` to `player`. If that post ultimately fails, the client restores the consumed bills — the items are the fallback record.

9.4 Job board — escrow and payout

```
Post:     job_post_escrow   player -> job     (poster charged, listing funded)
Approve:  job_payout        job    -> player  (worker paid)
Cancel:   job_refund        job    -> player  (poster refunded)
```

The escrow itself lives in the listing's NBT in the world save. The service sees only the two player-side movements, which is all it needs for the books to balance.

9.5 Administrative adjustment

```
POST /processTransaction
{ "type": "admin_credit", "amount": 50000,
  "source":      { "type": "system", "id": "faucet.admin" },
  "destination": { "type": "player", "playerUuid": "target-uuid" },
  "initiator":   { "playerUuid": "admin-uuid", "playerName": "AnAdmin",
                   "role": "admin" },
  "reason": "Admin grant" }
```

An absolute "set balance" uses `/setBalance` where available; otherwise the client reads the balance, computes the delta, and posts `admin_credit` or `admin_debit` — accepting a small race window that `/setBalance` with `expectedVersion` closes.

10 Reference implementation notes

Non-normative guidance for building the service.

- **Serialize per account.** Take a per-account lock (or a single global lock, entirely adequate at Minecraft-server scale) around read–validate–write. Balance checks that race produce overdrafts.
- **Persist the idempotency record in the same transaction as the balance change.** If they can diverge, a crash between them reintroduces the double-charge the key exists to prevent.
- **Make `version` a per-account counter** incremented inside that same critical section.
- **Append, never mutate.** Voids are new entries. A ledger's value is that it is a history, not a current state.
- **Store the raw type string.** Do not map `type` onto a closed enum in your data model, or the next client feature will require a service deployment.
- **Reject, don't clamp.** An amount you cannot represent should fail loudly with `AMOUNT_INVALID`; silently rounding creates discrepancies nobody can trace.
- **Return `balances` on `INSUFFICIENT_FUNDS`.** It halves the round trips in the most common failure.

A minimal conforming service is roughly: an accounts table (`accountId`, external identity, `balance`, `version`, `status`), an append-only transactions table, and an idempotency table keyed by `idempotencyKey` with the stored response. Everything else is optional.

11 Client configuration

How a client is configured is not part of the protocol — a service never sees it, and each client names its settings differently.

For SUM specifically, see `OPEN_MCECONOMIC_API_SETUP` alongside this document, which walks through the `economy_api` config category, TLS and certificate handling, and the deployment choices an operator has to make.

Whatever a client calls them, three behaviours are load-bearing enough to name here, because a service can be broken by a client getting them wrong:

- **The base URL MUST be `https`** in any deployment where client and service do not share a trusted private network (Section 5.4).
- **Only the logical server may hold the bearer token** (Section 3.2). A token shipped to a game client is a published token.
- **A client MUST declare its environment truthfully** (Section 4.2), even when an operator has deliberately enabled a configuration the service might refuse.

12 Conformance

A service is **conforming** if all of the following hold.

Endpoints

☐ `GET /health` returns currency, capabilities, `settledPartyTypes`, and limits.

☐ `POST /resolveAccounts` maps UUIDs to opaque account ids, one result per request entry, in order.

☐ `POST /getBalances` returns balances for a batch of ≥ 50 accounts.

☐ `POST /processTransaction` applies a movement atomically and returns post-transaction `balances`.

☐ `GET /getTransaction` resolves by `transactionId` *and* by `idempotencyKey`.

Money

☐ All amounts are JSON integers in minor units; no floats appear anywhere.

☐ `minorUnitDigits` is declared and honoured consistently.

☐ Amounts and balances fit in signed 64-bit integers.

☐ Non-integer, negative, or oversized amounts are rejected with `AMOUNT_INVALID`.

Idempotency

☐ Keys are persisted for ≥ 24 hours with their responses.

☐ A replay with a matching body returns the stored response with `replayed: true` and applies nothing.

☐ A replay with a differing body returns `409` / `IDEMPOTENCY_KEY_REUSED` and applies nothing.

☐ The idempotency record and the balance change commit together, or not at all.

Correctness

☐ Every transaction is atomic; a partial application is impossible.

☐ `version` is strictly increasing per account across every balance change.

☐ Debits that would overdraw are rejected with `INSUFFICIENT_FUNDS` unless `negativeBalances` is declared and an admin initiator sent `allowNegative`.

☐ The ledger is append-only; voids are new entries referencing the original.

Robustness and forward compatibility

☐ Unknown `type` values are accepted and recorded verbatim (unless `strictTransactionTypes` is declared).

☐ Unknown request fields are ignored, not rejected.

☐ Any header the service requires is published in `requiredHeaders`, and its absence is rejected with `MISSING_REQUIRED_HEADER` rather than a generic error.

☐ If `/validateTransaction` is offered, it mutates nothing — no balance, ledger entry, or idempotency record.

☐ Virtual party types are accepted.

☐ Every response — including errors — is JSON carrying the standard envelope.

☐ HTTP status codes agree with `error.code`.

Security

☐ Bearer tokens are validated on every request, including `/health`.

☐ The API is served over HTTPS with TLS 1.2 or newer; plaintext requests are rejected, not served.

☐ The certificate is valid for the host clients are configured with (self-signed is acceptable when operators are given the certificate to pin via `certificatePath`).

☐ The token never appears in a response body or a log line.

☐ Player-supplied text is stored as untrusted data and length-limited.

13 Change policy

- **Additive changes** — new optional endpoints, capability flags, response fields, and transaction types — do not bump the major version and **MUST NOT** break a conforming service.
- **New transaction types will be added** as clients grow. Services must tolerate them.
- **Breaking changes** require a new path version (`/v2`), negotiated via `/health` before use.

Changelog

Version	Date	Notes
`1.0`	31 July 2026	Initial specification.